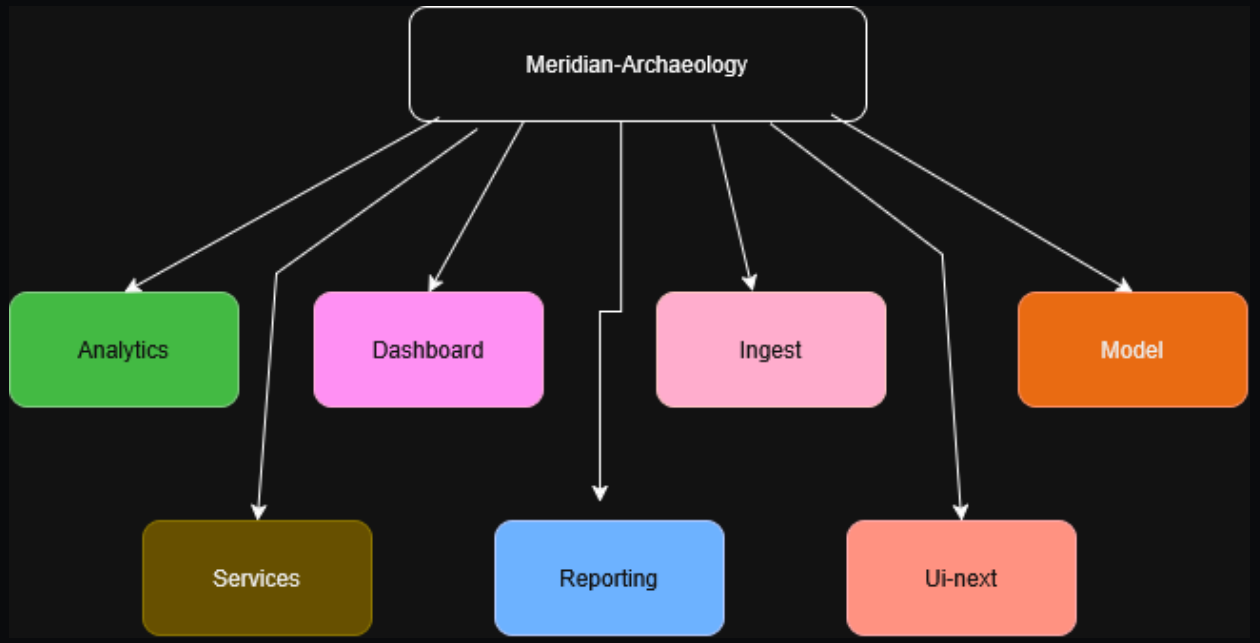
Group 5 – Almanac

Addison Broschell, Arianna Wallace, Harveen Harveen, Leo Goebel, and Aithan McIntyre

System Map



Subsystem	Language	Purpose
Analytics	Python	This subsystem is responsible for importing, exporting, summarizing data and exposes SQLite database and model files
Dashboard	Javascript, CSS and HTML	This subsystem is responsible for displaying the charts, tables and how the program looks as a whole. Dashboard exposes a web interface using API requests
Ingest	C	This subsystem is responsible for station constants, tail handling and accessing the store. This exposes binary telemetry frames.
Model	Fortran	This subsystem is responsible for simulating crop growth and soil water balance to generate forecasts. It exposes fixed width input deck and output files.
Reporting	Python	This subsystem is responsible for formatting, generating and testing reports. Reports and export files are exposed.
Services	Java	This subsystem is responsible for the admin tool, in memory cache and api. Services expose HTTP REST API (JSON) interface.
Ui-next	TypeScript/React	This subsystem is responsible for the navigation, layout and screens for the program. Ui also exposes web interface and API requests.

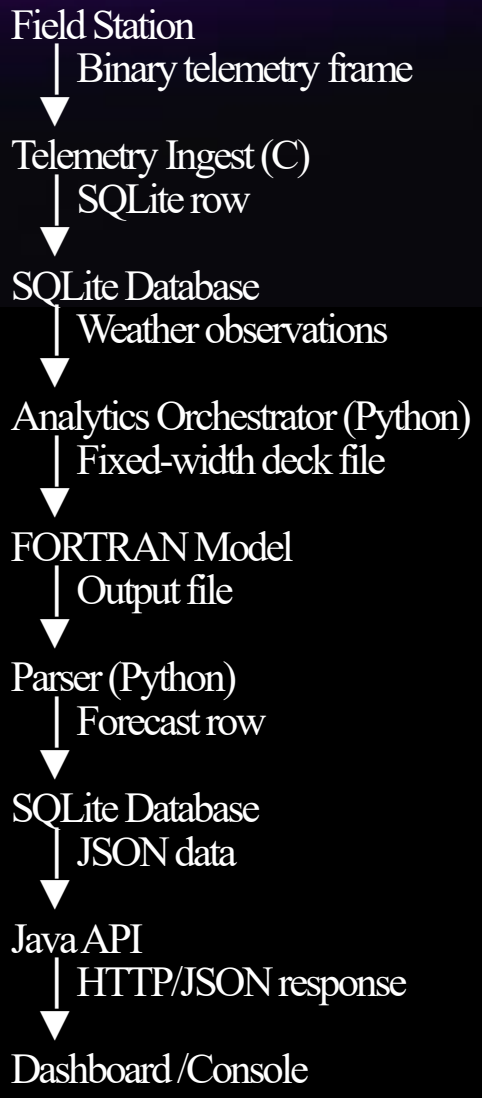
System Map

DATA FLOW: Data moves through the system as weather readings are collected, stored, processed and displayed. A binary telemetry frame is stored as a SQLite row, converted into a fixed-width deck for the FORTRAN model, transformed into forecast records, and finally sent as a JSON to the Dashboard display.

CONTROL FLOW: Control is managed by the Analytics Orchestrator, which coordinates reading weather data, running the FORTRAN model, and storing forecast results. When a user requests information from the Dashboard, the API retrieves the required forecast data from the database and returns it to the interface.

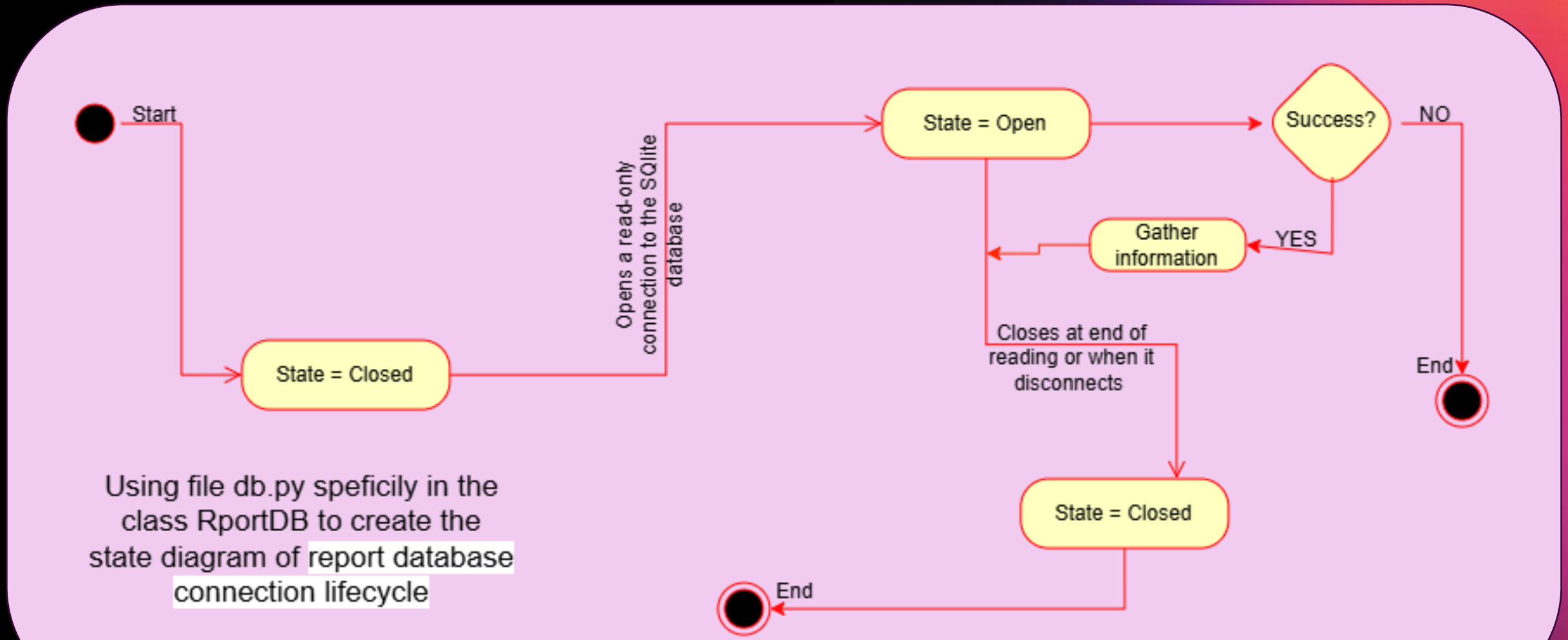
README Claim	Reality
the reporting subsystem replaced the cobol RPTGEN writer in 1997.	The git history shows the reporting subsystem being introduced in 2017 in commit a3298ea, whose commit message specifically says it replaces the cobol RPTGEN writer.
Meridian is a three-tier application which includes: ingest, services, and dashboard.	The current system is divided into seven subsystems rather than the three that are outlined in the README
In the README it states that the nightly batch is scheduled using Quartz and tells the developers to look at services/src/main/resources/quartz.properties.	The services directory does not contain the quartz.properties file that is stated in the README. The services directory contains Java source files instead. Quartz.properties does not exist

End to End Trace



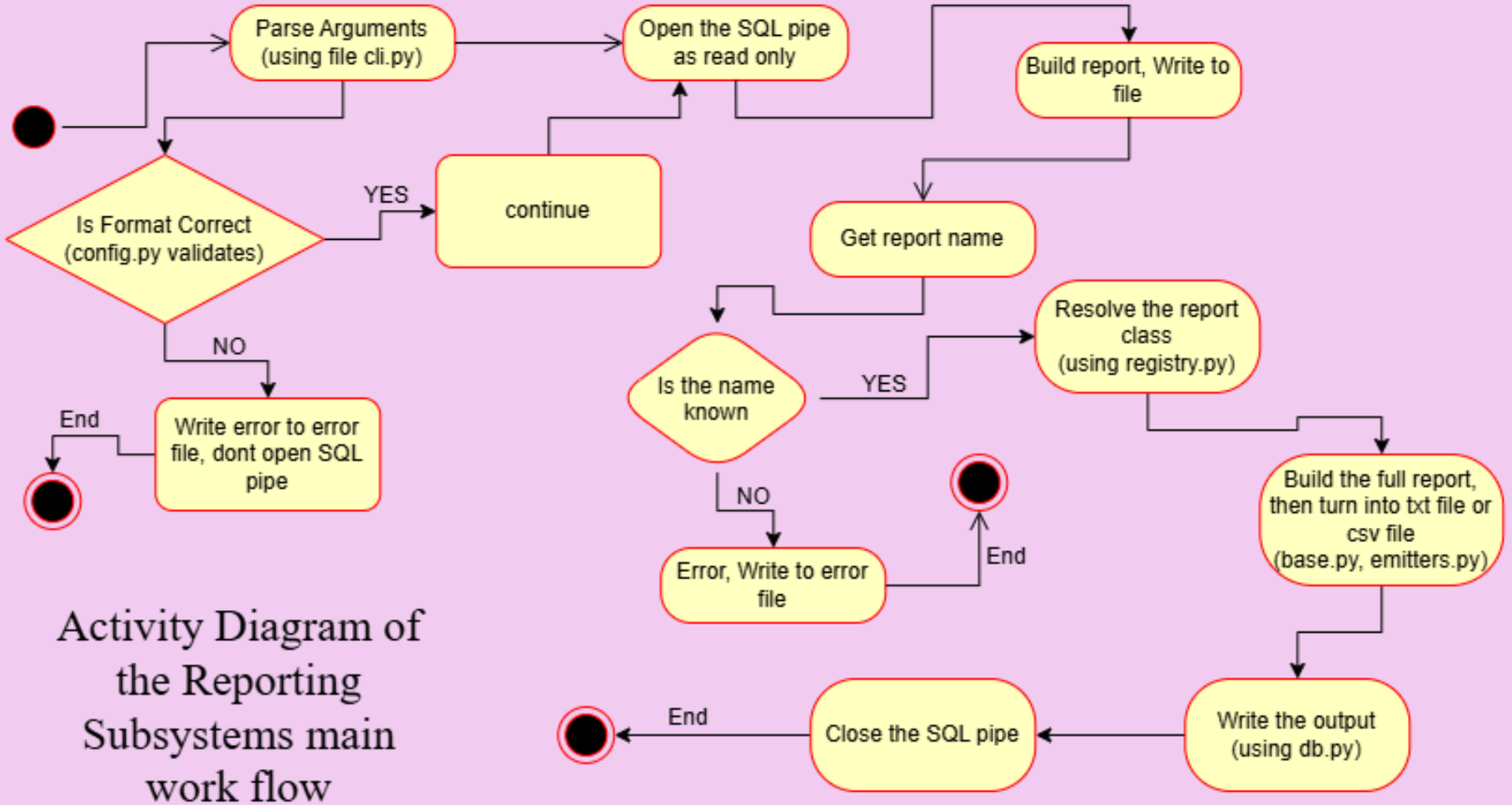
- Weather data starts at a field station as a binary telemetry frame.
- The telemetry Ingest subsystem validates and stores the reading in the SQLite database.
- The Analytics Orchestrator reads the data and prepares a fixed – width deck for FORTRAN Model.
- The FORTRAN Model generates forecast results, which are parsed and stores back in the database.
- The Java API retrieves forecast data and sends it to the Dashboard as JSON.
- The Dashboard displays the forecast values and charts to the user.

Behaviour of Subsystem State Diagram



Using file db.py speficily in the class RportDB to create the state diagram of report database connection lifecycle

Behaviour of Subsystem Action Diagram



Activity Diagram of the Reporting Subsystems main work flow

Behaviour of Subsystem Interface Contract

Inputs	Output	Invariants	Error Behaviour	Exact format and dependencies
season_summary.py → year, valid forecast station_report.py → station, valid forecast daily_detail.py → station, year anomaly.py → nothing mandatory coverage.py → forecast	Writing to the error file Creates a text file Creates a CSV file	Database is read-only, doesn't create tables Format is specified by config.py Fixed number format, column width	Everything is written to std.err Return exit code 1 Db.py raises ReportDBError or with anything is wrong with the api connection	Db.py needs reading, forecast and daily forecast tables Config.py need page width of 80 or 132 columns

Subsystem Biography

R. Halloran

Replaced COBOL RPTGEN Season + station + daily reports

2017

P. Adeyemi

The column width was updated from 80 columns to 132 columns

2021

P. Adeyemi

Coverage, anomaly reports, folded query helpers, and tests were added.

2023

Key Part

Two Different Column Layouts

NARROW

80-column layout to support historical MERDIAN.OUT

WIDE

132-column added for extra output space

Both versions are still supported for compatibility because a lot of the terminals never got wider than 80 columns.

Open Tickets In Subsystem

The only open ticket that touches our subsystem, Reporting.

ID	Date Opened	Component	Severity	Summary
MRD-238	2025-11-14	Reporting	Major	The report writer selects the current forecast per station with MAX(run_at) compared as text. A manual backfill written in a different timestamp format sorts wrong, so a report can print an older run as the one "in force".

The issue is case by line 211 in db.py within the definition of the lastest_forecast_for_station function.

The root cause of the issue is that The string being passed to the _one function extends to other lines that should be run as code.

This issue touches functions in station_report.py and test_reports.py. As well as the Station Report.

Here Be Dragons

1. Dragon:
Fixed width reporting format

Where:
reporting/reporting/README.md,
format/numfmt.py and format/columns.py

Why it matters:
The reporting system depends on the exact column widths so current reports line up with older archived reports. Changing spacing, widths or alignment would shift the entire report layout. Numbers that are too large are replaced with asterisks rather than being allowed to change the column width.

What to know:
Do not change column widths, spacing or alignment, or overflow behaviour without checking how it affects the full report format.

STATION	YEAR	YIELD	RAIN
GUELPH	2021	123.4	456.7
WATERLOO	2021	******	23.1

2. Dragon:
Latest forecast depends on timestamp text formatting

Where:
reporting/reporting/db.py,
latest_forcast_forstation() and latest_run_at()
(This is also documented in issue MRD-238.)

Why it matters:
The system uses MAX(run_at) to find the newest forecast, but run_at is stored and compared as text. If timestamps are entered using different formats, the text sorting can be wrong and an older forecast could be reported as the newest one.

What to know:
Any code that writes or changes run_at values needs to keep the timestamp format consistent. Before changing how timestamps are stored or compared, check any part of the system that relies on the current format.

run_at (TEXT)	
2021-01-15 10:00	✓
2021-1-5 9:00	✗
2021-01-05 09:00	✗



3. Dragon:
Stations with no forecasts can disappear from the coverage report

Where:
reporting/reporting/db.py in `all_stations_ids()`
and reporting/reporting/reports/coverage.py

Why it matters:
The coverage report builds its station list from stations already found in the forecast table.
A station with no forecasts at all can therefore be left out instead of being shown as missing.

What to know:
The station list used by the coverage report is not necessarily the complete list of expected stations. Check the canonical station list before assuming that the stations returned from the database represent every station that should exist.



THANK YOU FOR LISTENING